

Review

Non-linear optimization of track layouts in loop-sorting-systems

Søren E. Sørensen ^{a,*}, Michael R. Hansen ^b, Morten K. Ebbesen ^b, Ole Ø. Mouritsen ^a^a Department of Mechanical and Manufacturing Engineering, Aalborg University, Pontoppidanstræde 105, 9220 Aalborg Ø, Denmark^b Department of Engineering, University of Agder, Grooseveien 36, 4876 Grimstad, Norway

ARTICLE INFO

Article history:

Accepted 11 November 2012

Available online 23 December 2012

Keywords:

Collision detection

Design optimization

Relaxed problem

Multi-stage approach

ABSTRACT

Optimization used for enhancing geometric structures is well known. Applying obstacles to the shape optimization problem is on the other hand not very common. It requires a fast contact search algorithm and an exact continuous formulation to solve the problem robustly. This paper focuses on combining shape optimization problems with collision avoidance constraints by which a collision detection algorithm is presented. The presented method is tested against the commercial loop-sorting-system used for sorting of medium sized items. The objective is to minimize price and footprint of the system while maintaining its functionality. Contact constraints are in this context important to include as various obstacles may surround the loop-sorting-system.

Crown Copyright © 2012 Published by Elsevier B.V. All rights reserved.

Contents

1. Introduction	19
2. The topology of LSS	20
3. Formulation of the optimization problem	21
3.1. Parametric model of the track layout	22
3.2. Objective function	22
3.3. Geometric constraints	22
3.4. User defined constraints	22
3.5. Non-contact constraints	23
3.6. Obtaining the discrete solution	24
4. Optimization approach	25
4.1. The gradient solver	25
4.2. The non-gradient solver	25
5. Results	25
5.1. Case #1	25
5.2. Case #2	26
5.3. Case #3	28
5.4. Discussion	28
6. Conclusion	29
Acknowledgments	30
References	30

1. Introduction

Optimization problems involving geometric design variables and objectives like compliance, weight, durability, etc. have been known for decades as strong tools for enhancing the performance in building structures

and industrial products. Shape optimization has been widely studied, see [1], [2] and [3] and has been utilized for enhancing various kinds of problems, like for example the blades of wind turbines, see [4] or the shape of a turbine disk, see [5]. Furthermore, the method of topology optimization [6] has revolutionized the way of designing spatial light weight structures with similarities to structures known only from nature.

Several geometric optimization tools handle even very complex shapes, however they often lack the ability to include obstacles. A method to include obstacle constraints in the optimization problem is the use of collision detection algorithm. The problem of geometric optimization in

* Corresponding author.

E-mail addresses: Ses@m-tech.aau.dk (S.E. Sørensen), Michael.R.Hansen@uia.no (M.R. Hansen), Morten.K.Ebbesen@uia.no (M.K. Ebbesen), Oom@m-tech.aau.dk (O.Ø. Mouritsen).

which a collision algorithm is utilized to avoid obstacles of any arbitrary shape is dealt with in this paper.

In literature some research considers the combination of optimization and collision detection algorithms. An example of an optimization problem combined with collision detection algorithms is a five-axis milling machine in [7] where emphasis is put on obtaining an enhanced surface quality. The presented method utilizes a gradient based approach by minimizing the movement generated by each rotation axis of the machine. A collision detection algorithm for multi-axis NC-machining is presented in [8]. Contacts are identified by searching through a set of polygons defining the surface of each obstacle by representing the tool profile using a ray tracing technique.

In robotic motion optimization techniques are employed to obtain a collision-free trajectory, [9]. The optimum trajectory is obtained by minimizing smoothness and accelerations between two pre-specified trajectory points. Collision is included by the use of a penalty formulation. Ratliff et al. present convincing results of the optimal spatial trajectory for a robotic arm and a quadruped robot using a gradient based optimization solver.

Aircraft trajectory planning is addressed as a planar optimization problem in [10]. Dynamics are modeled using a point mass formulation subject to constraints of maximum speed and force acting upon the aircraft. Collision avoidance constraints are imposed by an exclusion region around the vehicle that forces the aircraft to turn when approaching an obstacle. The optimal trajectory is obtained by minimizing the time of reaching a predefined waypoint.

The combination of collision detection algorithms and optimization problems is also addressed in the field of optimal part layout.

A layout optimization method of machining units is presented in [11] utilizing a simulation model to address both cost and time of the material handling process. The location of each machining unit is obtained by a slicing technique using the Genetic Algorithm to find the optimal solution.

A corresponding layout problem is addressed in [12] by optimizing the location of components in a shelter cabinet taking the mass distribution, the energetic network and the components' accessibility into consideration. The multi-objective problem is solved using the Genetic Algorithm by which the overlapping areas of the intersecting bodies are used as contact constraints.

A five stage approach to determine the construction site layout is presented in [13]. The multi-objective location problem of the construction site considers the interaction relationship in terms of material flow, personal flow, equipment flow, safety/environmental concerns and users' preferences. The multiple solutions generated by the optimization problem are evaluated in a selection stage together with the user.

Also, a two-stage strategy for layout optimization of satellite modules is presented in [14]. Collisions are defined as volumetric constraints of the overlapping volumes among boxes and cylindrical shapes with the aim of minimizing the residual between the desired and the current moment of inertia. Convincing results from two layout problems are presented while testing five different non-gradient optimization algorithms.

The paper revolves around the issues of combining collision algorithm with an optimization problem. The combination of an optimization problem and collision algorithms often proves heavy to compute due to the costly search of collisions and due to the number of iteration necessary to find the optimum. As a consequent the collision detection algorithm is often reduced to handle only simple geometries. The development of an improved contact search formulation is presented in this paper. The problem attaining an optimal track layout in Loop-Sorting-Systems (LSS) is performed to demonstrate the applicability of the presented method.

The commercial product of LSS is used for sorting of medium sized items like luggage and postal packages. It consists of a closed loop track in which a chain of carts transport items from an induction area to a predefined destination along the track. The track may be as long as 2 km and shaped using standard elements to facilitate the sorting task regarding the space limitations.

Today the layouts of LSS are developed manually by the design engineers. The manual process induces a development period that requires a limited number of design iterations to reach a satisfying solution. Because of a limited number of design iterations the final layouts are primarily fulfilled with regard to the demands of functionality while other competitive parameters like price and footprint are assigned less attention. An obvious choice of objective function would therefore be the price and footprint while maintaining the functionality of the LSS through constraint formulations.

A method for optimization of chain performance in LSS has previously been presented in [15] and [16]. Assuming dynamic vibrations in the LSS to originate from the polygon action within the chain of carts, Ebbesen et al. minimize the global polygon action using a kinematic simulation model. The polygon action is a forced velocity variation evolving from the discrete chain when it transits from going in a straight direction to becoming a chord of a circle arc as it changes direction in a curve, see [17]. An inherent formulation consisting of track lengths and radii of the curves is utilized as design variables to parameterize the track layout. The parametric formulation proves efficient for the simple test case presented in [15] and [16] although the inherent variables introduce a number of equality constraints in order to maintain a closed loop track. Minimizing the polygon action does, however, only represents the dynamic performance. The work in [15] and [16] clearly shows that it does not change the layout dramatically. This makes it suitable as a final optimization of LSS but it is not useful to determine the overall geometry.

The objective of the presented research has been to develop a collision algorithm which proves robust and fast to utilize. By robust is meant an algorithm which always provides a smooth continuous single value and maintains a convex design space as far as possible. Simultaneously the algorithm must prove fast in order to minimize the time of solving the optimization problem.

The develop collision detection algorithm is presented for the commercial application of loop-sorting-systems. Yet, its relevance is found in similar applications like the layout of the chain on tracked vehicles, the layout of chain dives, conveyor layout, etc. The presented collision detection formulation may furthermore be reformulated to fit problems like building layout, pipe layout, ventilation layout, etc.

2. The topology of LSS

LSS are used for medium sized items by which the main business segments are sorting of luggage and postal packages. LSS can handle items varying in size from a letter to a refrigerator and in mass from 0.1 kg up to 50 kg. The loop-sorter responsible for the actual sorting is the key component of LSS, see Fig. 1. The loop-sorter consists of a track forming a closed loop in which a chain of carts drive with a steady speed between 1.5 m/s and 3.0 m/s. The track is customer defined using eleven different standard track segments that may be divided into the following categories: straight tracks, horizontal curves, vertical curves and spiral curves. Spiral curves are designed with both inlets and outlets to initiate or finalize a level change. The horizontal curves have the standard radii of 2000 mm, 3000 mm or 4000 mm and with a sweep angle which is a multiple of 2.5°.

The chain of carts is connected through spherical plan bearings and interfaces to the track through four wheels mounted on each cart. A set of linear motors are mounted along the track underneath the chain. They provide magnetic forces to propel the chain of carts without any mechanical contact.

A carrier is mounted on top of each cart which handles the transportation of items. Sorting is commenced by inducting items onto the carrier and subsequently discharging the items as they reach their defined destination along the track. Average LSS handle between 0.1 and 1.0 million items per day.

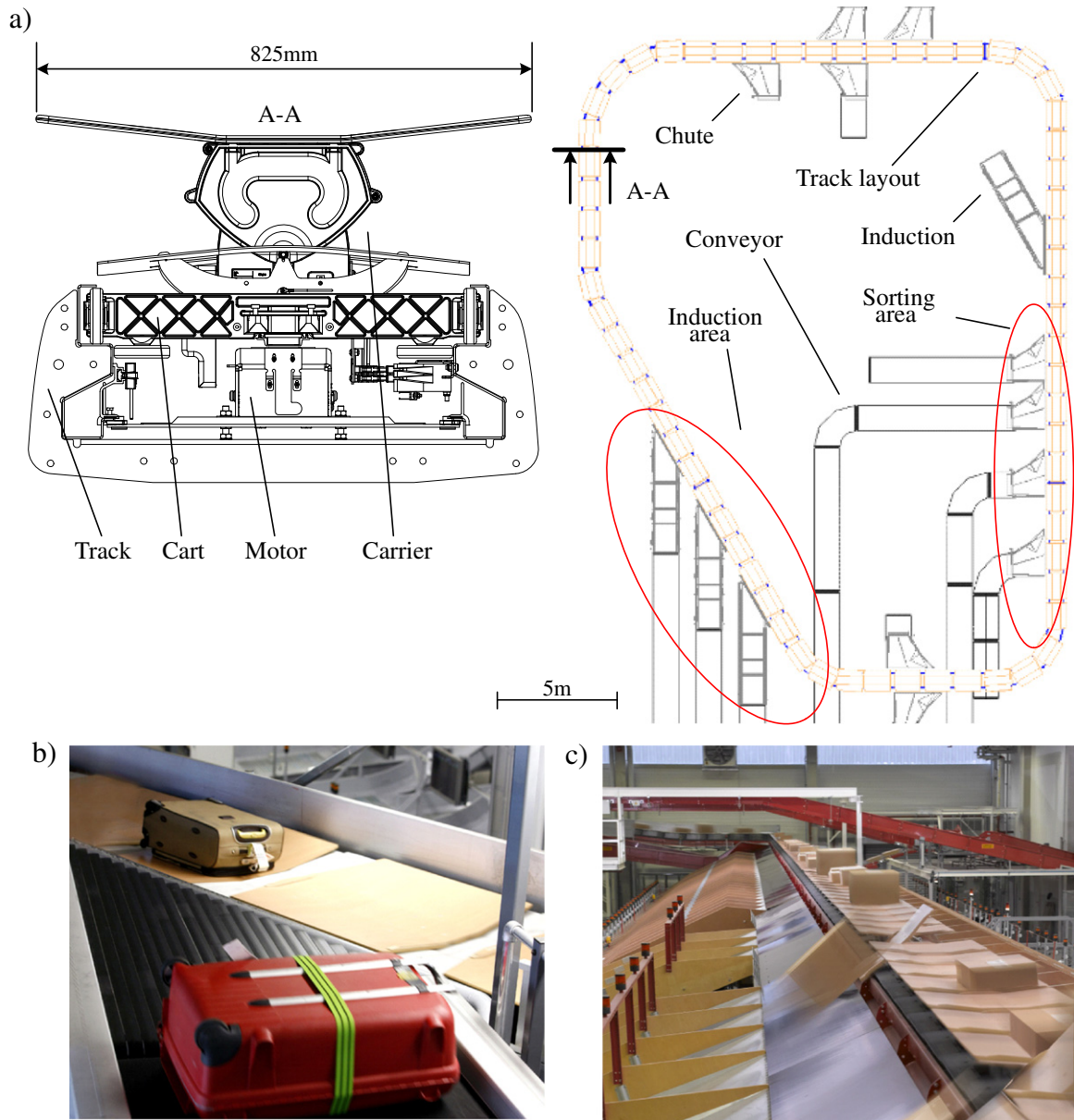


Fig. 1. The topology of LSS. a) Track layout with chutes and inductions. b) Cross-loading of items onto chain of carts. c) Sorting area with several chutes located along the track side.

3. Formulation of the optimization problem

The proposed layout optimization method is made to handle all shapes of planar track layouts. The optimization method uses a parametric model which defines the track layout via a vector, $\mathbf{x}$ of independent continuous design variables. This parametric model is further described in the following section.

The presented optimization problem is formulated as

$$\begin{aligned} & \text{minimize } f(\mathbf{x}), \mathbf{x} \in R^m \\ & \text{subject to } \mathbf{h}_i(\mathbf{x}) = 0, i = 1, \dots, m_1 \\ & \mathbf{g}_i(\mathbf{x}) \leq 0, i = 1, \dots, m_2 \\ & \mathbf{a} \leq \mathbf{x} \leq \mathbf{b} \end{aligned} \quad (1)$$

in which $\mathbf{h}_i(\mathbf{x})$ and $\mathbf{g}_i(\mathbf{x})$ are implicit equality and inequality constraint functions and the vectors $\mathbf{a}$ and $\mathbf{b}$ are the lower and upper bounds for the design variables. The objective function $f(\mathbf{x})$ is formulated as the track length which can be directly linked to the price and

footprint of the track layout. The implicit constraints are divided into three different types:

- Geometric constraints which ensure the track layout to be within a feasible shape.
- User defined constraints which restrain the layout in order to maintain the functionality of the LLS.
- Non-contact constraints to avoid intersection with the building or obstacles.

The optimization is carried out in three stages. A separate optimization problem is treated in each stage based on the result of the previous. For each optimization additional constraints are included by:

1. Minimizing the track length subject to user defined and building collision constraints.
2. Including obstacle constraints.
3. Applying the penalty function to obtain a discrete solution of the relaxed problem.

The three stage strategy is adopted in order for the track layout to pass obstacles and finally to reach a discrete solution. The implicit geometric constraints and the explicit constraints of the design variables are applied at all three stages. The collision constraints, the obstacle constraints and the penalty formulation are described in the following sections.

The spatial topology of the track layout is reduced to a planar optimization problem. The use of a planar formulation is found reasonable as level changes of the track layout are pre-defined where as it do not affect the length of the track. Consequently, in layouts with level changes it is important to be aware of out of plan collision.

3.1. Parametric model of the track layout

An independent parametric formulation of track layouts has been developed to avoid the use of equality constraints between the design variables like in [15] and [16].

First of all the discrete curve radii and angles are relaxed to get a continuous formulation by which $\{R_i \in \mathbb{R} | 1900 \text{ mm} \leq R_i \leq 4100\}$ and $\alpha_i \in \mathbb{R}$. In which R_i is the radius and α_i is the sweep angle of each curve i .

The parametric formulation is defined by a set of n independent vertices $P_1, P_2, \dots, P_n$ by which the line segments $P_2P_1, P_3P_2, \dots, P_nP_{n-1}, P_1P_n$ connect each point into a closed polygon, $\mathbf{T}$, see Fig. 2. Each vertex is located in the intersection point between the neighboring straight sections where the track layout is defined by a set of continuous Euclidian coordinates. Curve sections located between the neighboring straight tracks are defined by a fillet radius by which each vertex encompasses three design variables $P_i(x, y, R)$.

With the presented method a track layout is parameterized without the use of equality constraints as all design variables are independent. Geometric constraints are on the other hand introduced in order to avoid infeasible combination of the design variables.

One infeasible combination is in cases where the angle of the curves is above $\angle 90^\circ$. Here, the angle between two neighboring straight sections becomes acute and the intersection vertex approaches the infinite. Curves with an angle above $\angle 90^\circ$ are therefore split into two curves using two vertices to parameterize the shape of the track, see Fig. 2. Similarly, curves with an angle above $\angle 180^\circ$ are split into three vertices. The division of large sweep angles is only performed at the parametrization

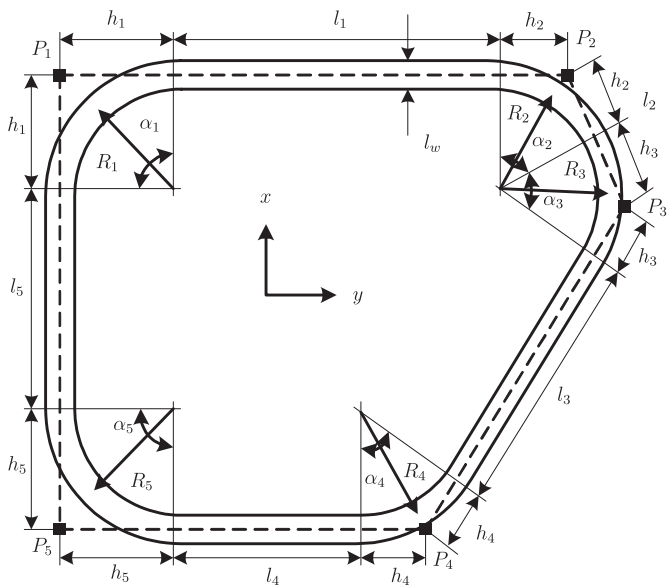


Fig. 2. Vertices of a closed polygon are used as design variable of the track layout together with the radii of the curves.

whereas in the optimization this is avoided through the geometric constraints.

3.2. Objective function

The objective of the optimization problem is to reach an enhanced track layout with reduced price and footprint while maintaining the sorting functionality of the system. From the utilized quotation method of today it is known that footprint and price of the layout are linear dependent on the track length. Therefore, a qualified measure of both price and footprint is the length of track. The objective function is formulated, based on the parametric formulation of the track layout in Section 3.1, as

$$f(\mathbf{x}) = \sum_{i=1}^n |\mathbf{d}_i| - h_i - h_{i+1} \quad (2)$$

in which the vector $\mathbf{d}_i$ is defined as

$$\mathbf{d}_i = \begin{bmatrix} x_{i+1} - x_i \\ y_{i+1} - y_i \end{bmatrix} \quad (3)$$

and the arc line h_i is obtained by

$$h_i = R_i \cdot \tan\left(\frac{\alpha_i}{2}\right) \quad (4)$$

The curve angle between the adjacent straight sections, α_i is defined as

$$\cos(\pi - \alpha_i) = (-\mathbf{u}_i) \cdot \mathbf{u}_{i+1} \quad (5)$$

in which $\mathbf{u}_i$ is the unit vector of $\mathbf{d}_i$. The experienced reader would notice that the equation is non-linear in respect with the design variables x_i, y_i and R_i which proves non-trivial to solve.

3.3. Geometric constraints

Two types of geometric constraints are used to avoid infeasible track layouts. The angle between the line segments is constrained to avoid acute angles below $\angle 45^\circ$ as $\lim h_i \rightarrow \infty$. Thus, the angle α_i is constrained by

$$0 \leq \alpha_i - \frac{3\pi}{4} \quad (6)$$

The distance between vertices is constrained to avoid curves from overlapping as

$$0 \leq h_i + h_{i+1} - |\mathbf{d}_i| \quad (7)$$

The geometric constraints are imposed on all vertices for all three stages of the layout optimization.

3.4. User defined constraints

In general, a number of practical considerations regarding induction and discharging sections along the track require that a number of equality constraints can be imposed by the user to maintain the functionality of the system. A total of five different equality constraints have been identified via an analysis of existing layouts.

The length of a straight track is constrained by

$$0 = l_i - l_i^0 \quad (8)$$

where l_i^0 is the prescribed length. Rotation of a straight track is constrained by

$$0 = {}_z \left| \mathbf{u}_i \times \mathbf{u}_i^0 \right| \quad (9)$$

in which $\mathbf{u}_i^0$ is the prescribed unit direction vector. The driving direction of the chain defines the axial direction of the track while the perpendicular direction denotes the radial direction. Hereby, the radial movement of a straight section is constrained by

$$\begin{aligned} 0 &= {}_z \left| \mathbf{u}_i \times \left(K_{i,1} - K_{i,1}^0 \right) \right| \\ 0 &= {}_z \left| \mathbf{u}_i \times \left(K_{i,2} - K_{i,2}^0 \right) \right| \end{aligned} \quad (10)$$

in which $K_{i,1}$ and $K_{i,2}$ are the starting and terminal points of the straight track defined by

$$\begin{aligned} K_{i,1} &= P_i + \mathbf{u}_i h_i \\ K_{i,2} &= P_i + \mathbf{u}_i (l_i + h_i) \end{aligned} \quad (11)$$

Likewise, the axial movement of the straight track is constrained by

$$\begin{aligned} 0 &= \mathbf{u}_i \cdot \left(K_{i,1} - K_{i,1}^0 \right) \\ 0 &= \mathbf{u}_i \cdot \left(K_{i,2} - K_{i,2}^0 \right) \end{aligned} \quad (12)$$

Eqs. (8)–(12) represent fundamental constraints that are combined to yield the following possible user imposed design constraints:

1. A fixed length obtained by Eq. (8).
2. A fixed length and rotation obtained by Eqs. (8) and (9).
3. A free movement in radial direction obtained by Eqs. (9) and (12).
4. A free movement in axial direction obtained by Eqs. (8) and (10).
5. All degrees of freedom are fixed and obtained by Eqs. (8), (10) and (12).

Obviously, other types of constraints may be formulated but it is believed that for practical purposes this can always be formulated as a combination of Eqs. (8)–(12).

3.5. Non-contact constraints

Non-contact constraints are introduced to avoid collision with the building walls and avoid various obstacles within the building. The non-contact constraints are made as an intersecting area constraint formulation by

$$0 = A_q \left(\mathbf{G}_d(\mathbf{T}), \chi_q \right) \quad (13)$$

in which A_q is the contact area, d is the left or right hand side of the track boundary, $\mathbf{G}$ and χ is the q 'th obstacle. A polynomial representation of body shapes is often used as polygon clipping algorithms like [18], [19] and [20] that compute the intersection of polygons very efficiently. However, the discretization of curves has shown to introduce discontinuities in the contact constraints. The discontinuities may be smoothed by refining the discretization, but this will be at the cost of computational efficiency.

To avoid discontinuities in the design space an algorithm that computes the contact areas analytically both for straight and curved sections has been developed.

An example of contact between an obstacle and one of the track sides is shown in Fig. 3. Obstacles are defined by closed polygons $\chi_q = O_{q,1}, O_{q,2}, \dots, O_{q,t}$ in which q is the object number and t is the number of vertices. All vertices of the closed polygons are arranged in clockwise order before commencing optimization.

The left and right hand sides of the track boundary $\mathbf{G}_d$ are defined by the two arrays $\phi_d = [S_1, S_2, \dots, S_n]$ of the straight sections and $\vartheta_d = [U_1, U_2, \dots, U_n]$ of the curved sections in which $S_i = (P_{d,i}, \mathbf{u}_{1,i})$ and $U_i = (C_i, R_{d,i}, \mathbf{u}_{1,i}, \mathbf{u}_{2,i}, D_i)$. The point $P_{d,i}$ is the starting point of the straight track side computed by

$$P_{d,i} = P_i - \mathbf{u}_i h_i \pm \hat{\mathbf{u}}_i \frac{l_w}{2} \quad (14)$$

where l_w is the width of the track and $\pm$ defines the left or the right hand side of the track respectively. The curve center and radius are C_i and $R_{d,i}$ whereas the unit vectors $\mathbf{u}_{1,i}$ and $\mathbf{u}_{2,i}$ define the direction of the starting and terminal points of the curve, see Fig. 3. The left or right direction of each curve is defined by $D_i \in [-1, 1]$.

The contact algorithm is divided into five steps, see Fig. 4.

Intersection search is initially commenced by comparing the line segments of χ_q against the straight sections ϕ_d and against the curved sections ϑ_d . Intersections are determined using the common theory for line–line and line–circle intersection, see [23].

Every point of intersection is classified as either ingoing, I_{in} or outgoing, I_{out} , relative to the overall driving direction, see Fig. 3. Accordingly, classification of each point is attained by the sign of the determinant between the two intersecting sections, as

$$C = {}_z |\mathbf{u}_o \times \mathbf{u}_t| \quad (15)$$

where $\mathbf{u}_o$ is the unit tangent vector of the obstacle in the contact point and $\mathbf{u}_t$ is the unit tangent vector of the track in the contact point. If C is positive the contact point is ingoing else if C is negative the contact point is outgoing.

All intersection points are assembled in the arrays, $W_{in} = [I_{in,1}, I_{in,2}, \dots, I_{in,m}]$ and $W_{out} = [I_{out,1}, I_{out,2}, \dots, I_{out,m}]$ in which m is the number of ingoing and outgoing intersection points corresponding to the number of contact areas.

If $m > 0$ the algorithm continues by computing the contact area(s). Each contact area is computed separately through three steps, see Fig. 4. In the first step the matching ingoing and outgoing intersection points of the contact area are identified. This is done by searching from the ingoing point $I_{in,j}$ along the direction of the track. The outgoing intersection point $I_{out,j}$ matching the ingoing point $I_{in,j}$ is defined by the point in W_{out} located closest to and in front of the ingoing point. The contact area between the matching intersection points $I_{in,j}$ and $I_{out,j}$ is the sum of the area of the enclosed polygon of the overlapping line segments and the area of each curve section penetrating the obstacle.

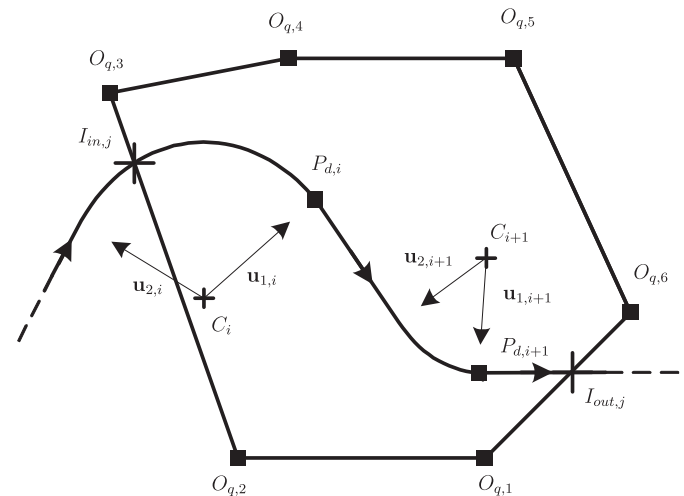


Fig. 3. Part of the track side in contact with an obstacle.

a local minimum for each discrete angle or radius. The penalty function is formulated as an equality constraint by

$$0 = \sum_{i=1}^n \left| \frac{\alpha_i}{\alpha_{de}} - \text{Round}\left(\frac{\alpha_i}{\alpha_{de}}\right) \right| + \eta \left| \frac{R_i}{R_{de}} - \text{Round}\left(\frac{R_i}{R_{de}}\right) \right| \quad (21)$$

in which α_{de} and R_{de} are the discrete multipla of $\angle 2.5^\circ$ and 1000.0 mm, respectively.

A power exponent utilized in [21] and [22] is removed in Eq. (21) as an analysis of the design space reveals that several local minima representing non-discrete designs appear.

The penalty factor, $\eta=2$ is multiplied with the radii in Eq. (21). This factor has proved to be necessary as the strong nonlinear coupling between the radii and the angles may get caught in a situation by which the discrete solution is never reached. The penalty factor induces the optimization algorithm to initially favor a solution of the radii by which the discrete solution of the angles is obtained smoothly hereafter.

4. Optimization approach

The layout optimization problem is tried out on two solvers to test the robustness of the proposed method. The two solvers use a gradient and a non-gradient algorithm, respectively.

4.1. The gradient solver

The constraint multi-variable algorithm *fmincon* provided in MatLab is used on the layout optimization problem. The *fmincon* solver is configured to use the sequential quadratic programming method with each iteration solved by an active set strategy, see [24]. The objective and constraint gradients are evaluated using forward difference and the Hessian matrix is obtained by a quasi-Newton approximation using the BFGS formula, see [25].

Geometric, user defined and non-contact constraints are implemented as nonlinear constraints. However, a penalty formulation is used in stage 2 on the obstacle constraints in order to avoid constraint violations in the initial evaluation. The default settings of the stop criterions in *fmincon* are used throughout all performed optimizations.

4.2. The non-gradient solver

The *Complex* solver is an unconstrained optimization algorithm which is presented in [26] and further developed in [27]. The stochastic *Complex* method searches the design space by mirroring the worst design candidate around the centroid of a population of design candidates. The initial population of design candidates is randomly generated when commencing optimization. The size of the population should reflect the number of design variables and the complexity of the problem. A population size of $9 \cdot n$ has been used in this work where n is the number of independent vertices.

A penalty formulation is used for the unconstrained *Complex* solver to handle the constraint layout problem. Therefore, a reformulation of Eq. (1) is performed by

$$\text{minimize } f(\mathbf{x}) + \sum_{i=1}^{m_1} \rho_i (\mathbf{h}_i(\mathbf{x}))^2 + \sum_{i=1}^{m_2} \varrho_i (\max(0, \mathbf{g}_i(\mathbf{x})))^2 \quad (22)$$

where ρ_i and ϱ_i are penalty parameters of the constraint equations.

A few restrains are applied to the *Complex* solver to get feasible solutions. The initial random generated population introduces several unfeasible track layouts as sections between the vertices may be crossed or have too acute angles. Likewise, the randomly generated layouts may impose large violation on the user defined constraints. Therefore, each track layout violating the geometric constraints is

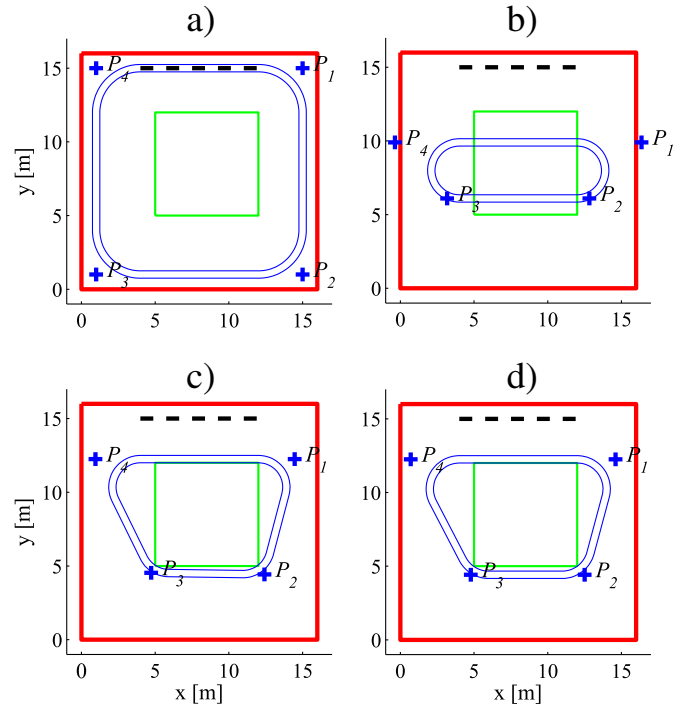


Fig. 6. Case #1 using the *fmincon* solver. a) Initial layout. b) Solution after stage 1. c) Solution after stage 2. d) Final solution.

replaced until a feasible design is reached. Also, the bounds of the Euclidian design variables are restricted to be within 5 m of the initial track layout at stage 1. The boundary restrains only concern the initial generated population as the evolution in new design candidates are allowed to move outside the boundary. The final population of stages 1 and 2 is used at the next stage as the initial population. To get a faster convergence each design variable in the initial population is perturbed with a random value between ± 0.5 m.

5. Results

The proposed layout optimization method is tested on three cases which all comprise user defined constraints and non-collision constraints.

The stochastic nature of the *Complex* solver entails that different solutions may be reached at each run. Especially stage 3 may encounter variation in the solutions as the design space holds multiple solutions. The *Complex* method is therefore performed ten times on each case.

5.1. Case #1

This simple case is a square track layout (blue lines) with four vertices corresponding to 12 design variables, see Fig. 6a). The building layout (red line) is surrounding the track layout and is likewise squarely shaped. One rectangular obstacle (green line) is located in the center surrounded by the track. The dotted line shows the applied

Table 1
The discrete solutions of case #1.

i	<i>fmincon</i>		<i>Complex</i> ^a	
	α_i (Deg)	R_i (m)	α_i (Deg)	R_i (m)
1	75.000	2.000	87.500	2.000
2	105.000	2.000	87.500	4.000
3	117.500	2.000	117.500	2.000
4	62.500	2.000	67.500	2.000

^a Best of ten runs.

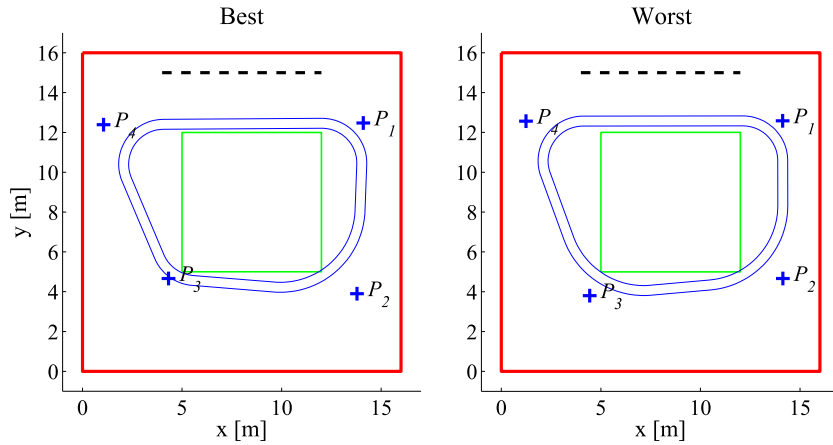


Fig. 7. Best and worst results of case #1 using the *Complex* solver.

user defined constraints. The straight track between the vertices P_4 and P_1 is applied user defined constraints by which it is only allowed to move in radial direction according to restrain type 3 described in Section 3.4.

Angles and radii of the final discrete solutions are shown in Table 1 for both the *fmincon* and *Complex* solvers.

The solutions of each stage using the *fmincon* algorithm are shown in Fig. 6. Obstacle constraints are not included at stage 1 whereas the track penetrates the square obstacle in the center, see Fig. 6b). At stage 2 the obstacle constraints are applied as a penalty function in which the track nestles close up to the obstacle. The discrete solution is found close to the result of stage 2, see Fig. 6d). Notice, that the user defined constraint between P_4 and P_1 is maintained in all three stages.

A discrete solution is reached in all ten runs using the *Complex* method. However, none of the solutions obtain the same shape. This proves the large number of local minima introduced by Eq. (21). The solutions reached by the *Complex* solver are despite variations in the layouts close to the length obtained by the *fmincon* solver. The best and the worst results obtained with the *Complex* solver are shown in Fig. 7.

The number of iteration and the reduction in length are shown in Table 4 for case #1. With the simple problem formulation in case #1 the *fmincon* solver has been able to reduce the length of the track with 33.5% and the *Complex* solver was able to reduce the length with 32.5%.

5.2. Case #2

A track layout from a real LSS is utilized in case #2. This layout is parameterized by 10 vertices corresponding to 30 design variables, see Fig. 8a). The building layout (red lines) is made of 6 vertices and is squarely formed with a small variation in the upper right corner. Three square obstacles (green lines) are located within the building layout. Three user defined constraints are applied to the track layout. Chutes used for discharging of items are located along the straight track segments between vertices P_2 and P_3 and between P_8 and P_9 . These straight sections are fixed in all degrees of freedom according to constraint type 5 described in Section 3.4. Likewise, induction of items is performed along the straight track section P_4 and P_5 . Consequently this section is not allowed to rotate according to constraint type 2.

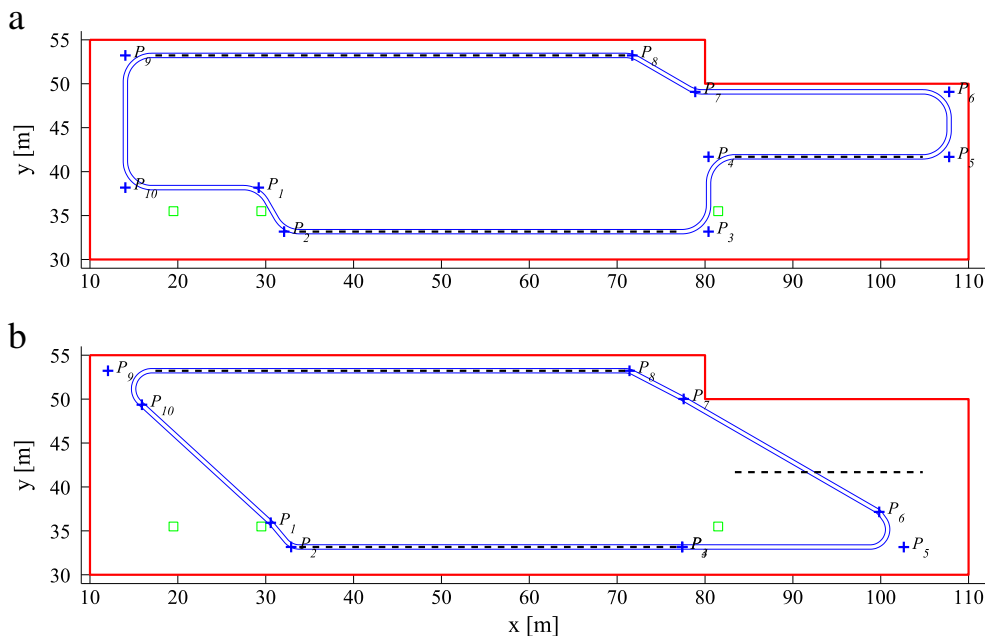


Fig. 8. Case #2 using the *fmincon* solver. a) Initial layout. b) Final solution.

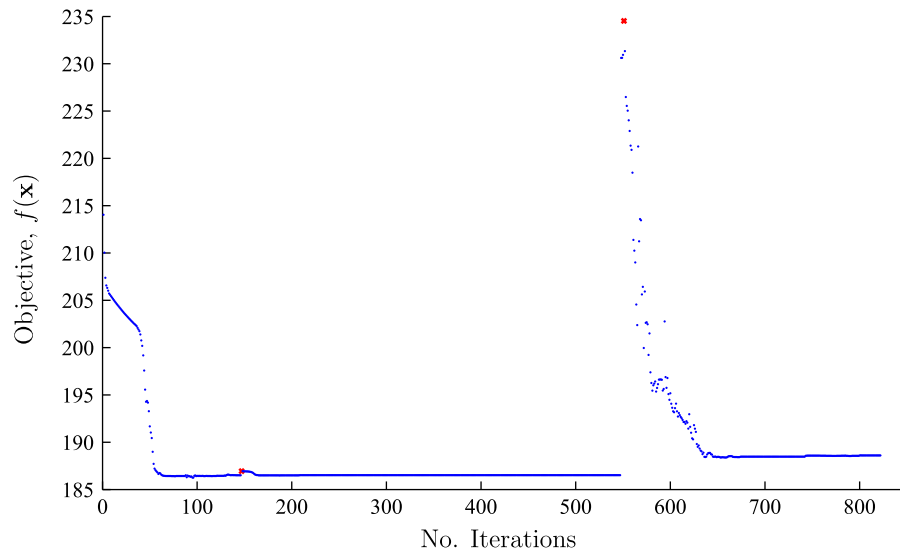


Fig. 9. Iteration history of case #2. A red cross indicates the start of a new optimization stage.

The solution obtained by the two solvers is shown in Table 4. The *Complex* solver manages to reduce the length of the track with a maximum of 11.9% while the *fmincon* solver reduces the track length with 12.5%.

The user defined constraints divide the layout into a left and a right hand problem by which the right hand side is the most complicated to solve. The elaborate right hand side arises from the constraint applied to the track section between P_4 to P_5 which causes the radius and the Euclidian coordinates of both points to become strongly coupled.

The final solution obtained by *fmincon* is illustrated in Fig. 8b). It is observed that the track between P_4 and P_5 has become tangent with the straight track between P_2 to P_3 in order to reduce the length of the track. The further entails that vertices P_3 and P_4 are located on top of each other. In such a case the presented optimization formulation proves efficient to handle this situation without reaching singularities.

Moreover it is observed that the sections between vertices P_9 and P_2 have been straightened out in order to reduce length.

The iteration history of case #2 using the *fmincon* solver is illustrated in Fig. 9 in which a red cross indicates the start of a new stage. The greatest reduction of the track length is obtained at stage 1. The small change in the first part of stage 2 emerges from an obstacle contact near vertex P_1 . The large number of iterations used at stage 2 is due to a small rearrangement between points P_{10} and P_1 . The scattered points at stage 3 evolve from the interdependence between angles as it is necessary occasionally to force one angle out of a discrete solution for the other angles to improve. It is observed that several iterations are without noticeable changes. The number of iterations might be reduced by changing the convergence criteria. However, the small changes are important in all three stages as they induce major changes in the position of the design vertices.

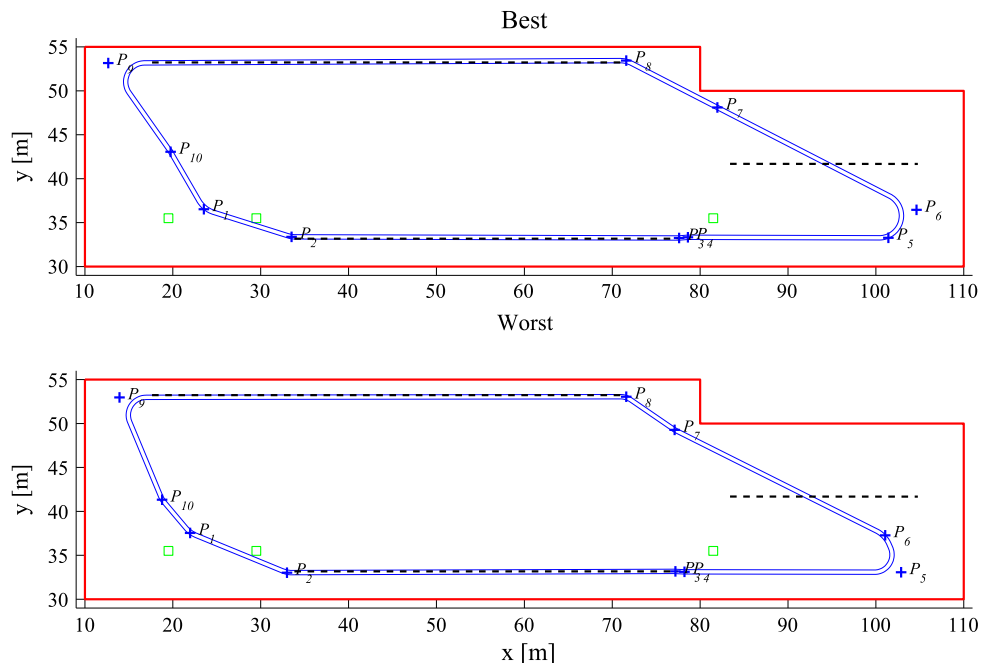


Fig. 10. Best and worst results for case #2 using the *Complex* solver.

Table 2
The discrete solutions of case #2.

<i>i</i>	<i>fmincon</i>		<i>Complex</i> ^a	
	α_i (Deg)	R_i (m)	α_i (Deg)	R_i (m)
1	172.498	2.000	155.000	3.505
2	130.013	2.002	159.661	3.887
3	179.992	2.000	177.508	2.000
4	179.990	2.000	177.385	2.010
5	54.996	2.000	45.222	2.028
6	154.995	2.000	165.000	3.001
7	117.500	2.000	175.000	4.000
8	177.497	2.000	154.974	4.000
9	45.000	2.065	65.021	2.068
10	177.485	3.000	160.000	3.973

^a Best of ten runs.

The best and worth solution obtained with the *Complex* solver is shown in Fig. 10. The results obtained by the *Complex* solver are close to the shape reached by the *fmincon* solver. Although, the left hand side of the track layout differs as the obstacle is avoided in the opposite direction.

The *fmincon* solver reaches an exact discrete solution while the *Complex* solver has several angles and radii that do not conform, see Table 2. The deviations from a discrete solution evolve from the user defined constraints and the strong nonlinear coupling between angles and radii. Clearly, the *Complex* solver is not well suited to handle the last important stage of obtaining a commercially feasible layout for this case.

5.3. Case #3

The track layout in case #3 is from an actual LSS, see Fig. 11a). The track layout is shaped like a banana and parameterized through 10 vertices. The building layout is composed of four vertices surrounding the track layout. Two obstacles are located close to the track within

the building layout. Both obstacles are rectangular and one of them is rotated 45°. The track is subjected to three user defined constraints emphasized by the black dotted lines. The straight section between the vertices P_3 and P_4 is locked in all degrees of freedom while the track sections between P_8 and P_9 and between P_{10} and P_1 are allowed to move in radial direction according to constraint type 3.

The results reached by the two solvers on case #3 are shown in Table 4 by which both solvers are able to reduce the track length with 12.9%.

The *fmincon* solver reaches a fine solution in all three stages of the optimization problem, see Fig. 11d).

The best and worth solution obtained by the *Complex* solver is shown in Fig. 12. None of the ten solutions reached by the *Complex* solver are identical which again shows the non-convex design space introduced by the discrete constraint function. It is notable in Table 4 that the best solution obtained with the *Complex* method is close to the *fmincon* solution despite the difference between the final layouts. This indicates a complex design space where several distinctly different designs have approximately the same performance.

The discrete solution of the curve radii and sweep angles obtained by the two solvers is shown in Table 3. The results obtained by the *Complex* solver differ from an exact discrete solution like in case #2 in which two angles vary from the multiple of 2.5°.

5.4. Discussion

The layout optimization problem is handled robustly by both solvers. However, the *Complex* solver clearly has difficulties in finding an exact solution to the discrete problem in stage 3. It is believed that the user defined constraints in combination with the strong nonlinear coupling between angles and radii yield a type of problem which the *Complex* solver is not well suited to handle. Several tests with different formulations of the discrete constraint function in Eq. (21) have been conducted without the *Complex* solver being able to robustly return a discrete solution.

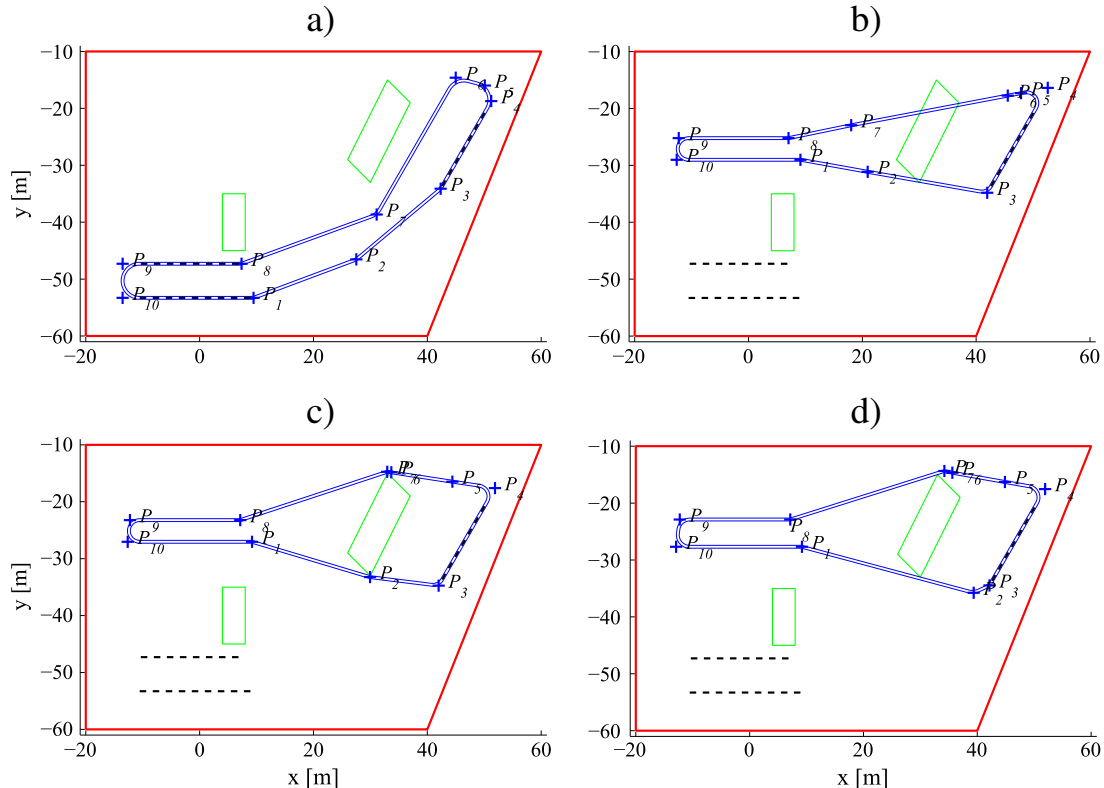


Fig. 11. Case #3 using the *fmincon* solver. a) Initial layout. b) Solution after stage 1. c) Solution after stage 2. d) Final solution.

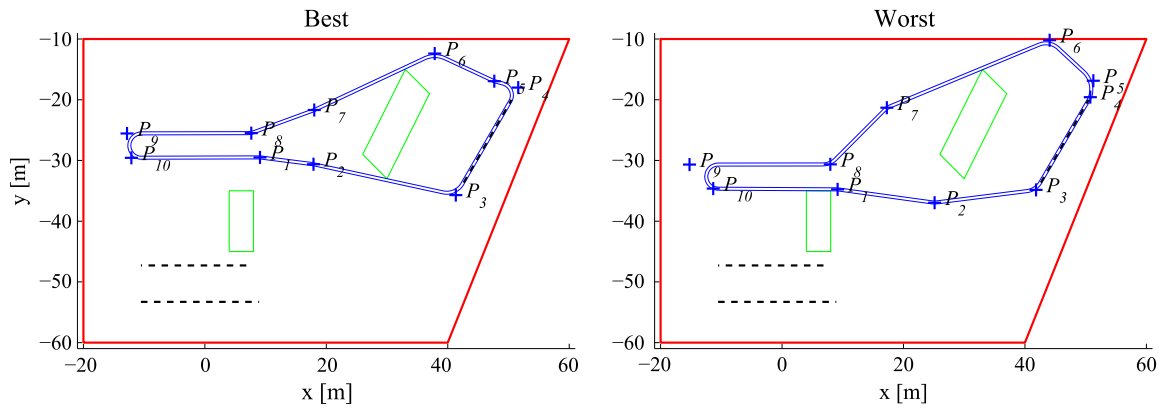


Fig. 12. Best and worst results for case #3 using the *Complex* solver.

Investigations of the design space in the third optimization run reveal several local optimums which emerge from Eq. (21). If the complex method does not reach a nearly exact optimum at the second optimization run a local optimum may be reached at the third optimization run. The global optimum may however be found by introducing a larger population in the complex algorithm or by applying another formulation to find the discrete solution. These options have however not been investigated any further.

The contact formulation works as anticipated. None of the solutions obtained by the *fmincon* solver intersect neither the building layout nor any of the obstacles. Small penetrations of the track sides are observed with the *Complex* solver as the external penalty method allows contact in return of reduced track length. Tests of the contact formulation reveal that a track section may get caught in an obstacle if the contact areas are equal on both sides of the track. This has been observed on a few occasions although it is not believed to be a problem in general.

The presented optimization method could be extended to a spatial formulation. Applying a spatial formulation will mainly have consequences for the parametric formulation and the contact search algorithm. The number of design variables will increase in the parametric formulation as the Z coordinate of each vertex must be added. With the additional Z-coordinate a formulation of the curve shape at each vertex also is to be reformulated. If a planar curve radius is still utilized as design variable a helix formulation may be used to describe the curve shape. If not a formulation of splines is an option.

With a spatial formulation the contact formulation also has to be modified from an area constraint formulation to a volumetric formulation. Additional considerations of the track geometry and the obstacle shape have to be made along with an efficient method of finding the overlapping volumes.

It is no surprise that the spatial formulation will increase the computational cost significantly. In addition, the complexity of the design

space will be more complex than in the planar formulation which might prove hard for the solver to handle.

6. Conclusion

A method of shape optimization using contact search algorithm has been put forward. The presented method proves strong in avoiding obstacles while still solving the objective. The commercial Loop-Sorting-System has been utilized as test case for the presented method with the objective of minimizing the price and footprint of the track layouts maintaining functionality through imposed constraints. The multi stage optimization method proves to be a useful tool in minimizing track lengths and thereby reducing price and footprint of the track layout.

A parametric formulation of the track layout has been developed to facilitate optimization. The parametric formulation utilizes the intersection point of the straight track sections and the radii of each curve as design variables. The proposed parametric model has proved to be efficient by applying a set of independent design variables.

Demands to the sorting capacity are maintained by imposing user defined constraints on the initial track layout. Five different types of user defined constraints can be assigned to the straight track sections to maintain the functionality of the LSS. The user defined constraints have appeared to be robust without introducing an over constraint problem. Inequality constraints might be implemented to improve the flexibility of the user defined constraints further.

An efficient and analytical procedure for computing any contact areas caused by intersections between track and obstacles has been developed and implemented in the optimization.

Three optimization stages are utilized to attain the minimum track length, to pass obstacle and reach a discrete solution. The use of three optimization stages proves efficient however it introduces numerous design iterations.

Tests of the optimization method have been performed on three test cases which characterize the overall variations in track layouts. Two solvers have been used to test the optimization method. Results reveal that the gradient based *fmincon* solver is superior in finding an optimal solution in all three test cases. The non-gradient based *Complex* solver

Table 3
The discrete solutions of case #3.

i	<i>fmincon</i>		<i>Complex</i> ^a	
	α_i (Deg)	R_i (m)	α_i (Deg)	R_i (m)
1	165.000	1.999	167.500	3.000
2	140.000	3.000	172.463	4.004
3	145.000	3.000	115.000	1.997
4	70.001	1.997	47.523	2.000
5	180.000	2.000	167.488	2.999
6	177.499	3.000	152.500	2.002
7	150.000	1.939	125.002	3.000
8	162.500	2.000	152.500	2.002
9	97.499	1.999	115.000	2.000
10	82.500	2.090	65.000	2.000

^a Best of ten runs.

Table 4
Results of the three cases with the *fmincon* and *Complex* solvers.

	<i>fmincon</i>		<i>Complex</i>					
	Length reduction	Number of iterations	Length reduction			Number of iterations		
			Average	Max.	Min.	Average	Max.	Min.
Case 1	33.5%	232	32.0%	32.5%	31.5%	14,434	17,830	8781
Case 2	12.6%	822	10.5%	11.9%	6.8%	71,909	85,270	54,160
Case 3	12.9%	1148	11.2%	12.9%	9.3%	81,463	99,780	52,490

reduces the track lengths however it lacks the ability to find a discrete solution in two of the presented cases. Investigation has been conducted in order to find the answer to the variation between the two optimization methods. However, it has not been possible to prove the actual cause of the problem. As a consequence we therefore recommend the *fmincon* solver as the preferred solver for these types of problems.

Acknowledgments

We would like to thank the Danish Agency for Science, Technology and Innovation (www.fi.dk) for financing the Industrial Ph.D project “Automatic Design of Sorting Systems”. We would also like to thank *Crisplant A/S* (www.Crisplant.com) for being most kind in providing information and materials for the presented layout optimization problem.

References

- [1] J. Haslinger, R. Mäkinen, *Introduction to Shape Optimization: Theory, Approximation and Computation*, Society for Industrial and Applied Mathematics, 2003.
- [2] Ashok D. Belegundu, Tirupathi R. Chandrupatla, *Optimization Concepts and Applications in Engineering*, Prentice Hall, 2003.
- [3] G. Allaire, Shape optimization by the homogenization method, in: *Applied Mathematical Sciences*, 146, Springer Verlag, 2002.
- [4] L.S. Johansen, *Analysis and Optimization of Composite Structures Using Adaptive Analysis Methods*, Department of Mechanical Engineering Aalborg University, Denmark, 2009.
- [5] Erik Lund, John Rasmussen, A general and flexible method of problem definition in structural optimization systems, *Journal of Structural Optimization* 8 (1994) 86–92.
- [6] Uwe Schramm, Ming Zhou, Recent developments in the commercial implementation of topology optimization, in: *IUTAM Symposium on Topological Design Optimization of Structures Machines and Materials: Status and Perspectives*, 2006, pp. 239–248.
- [7] C. Castagnetti, E. Duc, P. Ray, The domain of admissible orientation concept: a new method for five-axis tool path optimisation, *Journal of Computer-Aided Design* 40 (2008) 938–950.
- [8] Oleg Ilushin, Gershon Elber, Dan Halperin, Ron Wein, Myung-Soo Kim, Precise global collision detection in multi-axis NC-machining, *Journal of Computer-Aided Design* 37 (2005) 909–920.
- [9] Nathan Ratliff, Matt Zucker, J. Andrew Bagnell, Siddhartha Srinivasa, CHOMP: gradient optimization techniques for efficient motion planning, in: *IEEE International Conference on Robotics and Automation (ICRA)*, May, 2009.
- [10] Arthur Richards, Jonathan P. How, Aircraft trajectory planning with collision avoidance using mixed integer linear programming, *American Control Conference* 3 (2002) 1936–1941.
- [11] Farhad Azadivar, John J. Wang, Facility layout optimization using simulation and genetic algorithms, *International Journal of Production Research* 38 (2000) 4369–4383.
- [12] Julien Bénabès, Fouad Bennis, Emillie Poirson, Yannick Ravaut, Accessibility in layout optimization, in: *2nd International Conference on Engineering Optimization*, September 6–9, Lisbon, Portugal, 2010.
- [13] Xin Ning, Ka-Chi Lam, Mike Chun-Kit Lam, A decision-making system for construction site layout planning, *Journal of Automation in Construction* 20 (2011) 459–473.
- [14] Bao Zhang, Hong-Fei Teng, Yan-Jun Shi, Layout optimization of satellite module using soft computing techniques, *Journal of Applied Soft Computing* 8 (2008) 507–521.
- [15] Morten K. Ebbesen, Michael R. Hansen, Niels L. Pedersen, Morten K. Ebbesen, Michael R. Hansen, Niels L. Pedersen, Design optimization of conveyor systems, in: *III European Conference on Computational Mechanics Solids, Structures and Coupled Problems in Engineering*, Lisbon, Portugal, 2006, pp. 5–8.
- [16] Morten K. Ebbesen, *Optimal Design of Flexible Multibody Systems*, Ph.D. Thesis, 2008, Department of Mechanical Engineering, Aalborg University, Denmark.
- [17] S. Mahalingam, Polygonal actions in chain drives, *Journal of the Franklin Institute* 265 (1958) 23–28.
- [18] Bala R. Vatti, A generic solution to polygon clipping, *Journal of Communication of the ACM* 35 (7) (1992) 56–63.
- [19] Günther Greiner, Kai Hormann, Efficient clipping of arbitrary polygons, *ACM Transactions on Graphics* 17 (2) (1998) 71–83.
- [20] Lucanus J. Simonson, Industrial strength polygon clipping: a novel algorithm with applications in VLSI CAD, *Journal of Computer-Aided Design* 42 (2010) 1189–1196.
- [21] Michael Rygaard Hansen, Torben Ole Andersen, Design optimization of hydraulically actuated lifting table using mapping techniques, in: *The Ninth Scandinavian International Conference on Fluid Power, SICFP’05*, June 1–3, Linköping University Press, Linköping, Sweden, 2005.
- [22] Thomas H. Langer, Brian B. Christensen, Ole Ø. Mouritsen, Michael R. Hansen, Optimization of front axle suspension system of articulated dump truck, in: *The 1st Joint International Conference on Multibody System Dynamics*, May 25–27, Lappeenranta University of Technology, Lappeenranta, Finland, 2010.
- [23] <http://mathworld.wolfram.com>.
- [24] MathWorks, <http://www.mathworks.com/help/toolbox/optim/ug/brnnoxz.html>.
- [25] Jasbir S. Arora, *Introduction to Optimum Design*, in: Elsevier Academic Press, United States, 2004, pp. 327–328.
- [26] M.J. Box, A new method of constrained optimization and comparison with other methods, *The Computer Journal* 8 (1965) 42–52.
- [27] T.J. Manetsch, Toward efficient global optimization in large dynamic systems – the adaptive complex method, *IEEE Transaction on Systems, Man, and Cybernetics* 20 (1) (1990) 257–261.